



## Leading University

### Department of Computer Science and Engineering

**Course Title:** Computer Graphics Sessional

**Course Code:** CSE-4262

**Assignment Title:** Block Puzzle Game: Design and Development Using  
Computer Graphics ([GitHub Repository Link](#))

#### Submitted To

**Name:** Ashfakur Rahman

**Designation:** Adjunct Lecturer

**Faculty:** Dept. of CSE

#### Submitted by

**Name:** Mohsin Shah

**Department:** Dept. of CSE

**ID No:** 0182310012101041

**Batch:** 61

**Section:** A

# 1. Introduction

Computer Graphics provides the techniques required to generate, manipulate, transform, and display visual objects using a computer. While these concepts can be studied through mathematical formulas and diagrams, an interactive application provides a practical way to understand how the concepts work together.

The selected project is a 2D Block Puzzle Game. The game provides a simple but meaningful environment in which graphics rendering, coordinate systems, transformations, clipping, color, animation, keyboard input, collision detection, and game-state management can be combined into one working prototype.

## Project Objectives

- Build a working 2D interactive game using Python OpenGL.
- Apply at least four Computer Graphics techniques in a real application.
- Understand how 2D coordinates and transformations control game objects.
- Implement collision detection and line clearing as game logic.
- Practice debugging, testing, documentation, and live demonstration.
- Connect Computer Graphics knowledge with the Game / Graphics career track.

## Part 1: Problem Discovery

### 1. Which career track have you chosen?

#### Track A: Software Job (local or abroad)

I have chosen **Track A** because I am interested in working in the software industry. Software development involves problem-solving, programming, user interaction, and building applications that respond to real-time input. For my Computer Graphics Sessional project, I decided to build a **Block Puzzle Game** using Python, PyOpenGL, and GLUT. This project allows me to practice important programming and computer graphics concepts such as coordinate systems, 2D transformations, shape drawing, color filling, collision detection, animation, keyboard interaction, and boundary checking.

### 2. What is your project idea?

My project is a **Block Puzzle Game** made with **Python, PyOpenGL, and GLUT**. The program provides a 2D puzzle board where different types of blocks fall from the top of the board. The player can move the blocks left and right, rotate them, move them downward, or drop them instantly.

The main objective is to arrange the blocks so that complete horizontal lines are formed. When a line is completed, it is removed from the board and the player receives points. The game also contains a score system, level system, line counter, next-block preview, ghost piece, pause option, restart option, and game-over condition.

The project uses a grid-based coordinate system to represent the game board. Each block is represented using smaller rectangular cells. The position of the falling block is changed using 2D translation, while rotation is implemented using the 2D rotation transformation. Collision detection prevents blocks from overlapping with existing blocks or moving outside the game board.

The main problem my project demonstrates is how computer graphics techniques can be combined with programming logic to create an interactive real-time 2D game.

### 3. Which Computer Graphics concepts will your project use?

| Technique                           | Where it is used in my project                                                                                       | Real-world connection                               |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <b>Line &amp; Shape Drawing</b>     | The game board, grid, rectangular blocks, borders, panels, and interface elements are drawn using OpenGL primitives. | 2D games, CAD systems, UI graphics, vector graphics |
| <b>2D Transformations</b>           | Translation is used to move blocks, and rotation is used when the player rotates a block.                            | Game engines, animation systems, CAD, robotics      |
| <b>Color Fill</b>                   | Different puzzle blocks are filled with different colors such as cyan, yellow, purple, green, red, blue, and orange. | Games, visualization, graphical interfaces          |
| <b>2D Rotation</b>                  | The falling tetrominoes can be rotated by 90 degrees using coordinate transformation.                                | Games, object modelling, animation                  |
| <b>Collision Detection</b>          | The program checks whether a moving block touches the board boundary or another fixed block.                         | Games, simulations, robotics                        |
| <b>Boundary Checking / Clipping</b> | Blocks are prevented from moving outside the game board.                                                             | Viewport boundaries, games, graphics applications   |
| <b>Animation</b>                    | Blocks automatically fall downward using the GLUT timer.                                                             | Games, simulations, animation                       |
| <b>Grid-Based Coordinate System</b> | The board is divided into rows and columns, and every block occupies one or more cells.                              | Tile-based games, maps, CAD grids                   |
| <b>Alpha Blending</b>               | A transparent ghost piece shows where the current block will land.                                                   | HUDs, games, overlays, visual effects               |

### 4. Two real-world systems using similar techniques

- **Tetris and other block-based games:** Block puzzle games use a grid coordinate system, falling objects, rotation, collision detection, line/row removal, scoring, and real-time keyboard input. My project applies the same fundamental computer graphics and programming concepts in a simplified implementation.
- **Game engines such as Unity and Unreal Engine:** Modern game engines use transformations, collision detection, animation, rendering, input handling, and coordinate systems to create interactive games. Although my project uses PyOpenGL and GLUT instead of a modern game engine, the basic graphics concepts are similar.

## Part 3: Reflection Report

### Q1. Where did you get most stuck? Explain the bug/problem and how you solved it.

My biggest problem was implementing **block rotation and collision detection correctly**. At first, a block could rotate near the edge of the board and some of its cells could move outside the valid game area. In some situations, a rotated block could also overlap with blocks that were already placed on the board.

The problem occurred because the blocks were stored using local coordinates, while their actual positions on the board depended on the current x and y coordinates of the falling piece. I solved this by creating a separate function to calculate the rotated block coordinates before applying the rotation.

For a 90-degree rotation, I used the transformation:

$$x' = -y$$

$$y' = x$$

After calculating the new coordinates, the program checks whether the rotated position is valid. If the position would cause a collision with the boundary or another block, the rotation is not applied. I also added a simple wall-kick mechanism that attempts to move the block one cell left or right when rotation is blocked near the boundary.

Another problem was detecting when a falling block reached another block or the bottom of the board. I solved this by checking the four cells of the falling block against the board before every downward movement. When downward movement is no longer possible, the block is locked into the board.

## Q2. What did this project teach you that classroom theory could not?

In class, I learned the mathematical formulas for translation and rotation, but this project showed me how those transformations work in an actual interactive application.

For example, the translation formula can be written as:

$$x' = x + Tx$$

$$y' = y + Ty$$

In my project, changing the x-coordinate moves the block horizontally, while changing the y-coordinate makes the block move vertically.

I also learned that rotation is not only a mathematical formula. The rotated coordinates must also be checked against the game board and existing objects. Therefore, graphics transformations often need to work together with programming logic such as collision detection and boundary checking.

The project also taught me about animation. Instead of manually changing the block position, I used the GLUT timer to repeatedly update the game. This creates the appearance of continuous falling movement.

Another important lesson was debugging. When a block moved incorrectly, rotated outside the board, or collided unexpectedly, I had to observe the graphical output and trace the coordinate calculations. This helped me understand the relationship between mathematical coordinates and what is actually displayed on the screen.

## Q3. How does this connect to your career track?

For **Track A (Software Job)**, this project gives me practical experience in programming, graphical user interfaces, event-driven programming, and problem-solving.

The project uses keyboard callbacks, a display function, a timer-based game loop, data structures, coordinate transformations, collision detection, and modular functions. These concepts are useful beyond computer graphics and can also be found in game development, simulation software, interactive applications, visualization systems, and graphical user interfaces.

The project also helped me practice writing organized and documented code. I can use this project as a portfolio project to demonstrate my understanding of programming and computer graphics concepts.

## Q4. If you had one more week, what would you add?

If I had one more week, I would add the following features:

- **Hold Block:** Allow the player to save the current block and use it later.
- **High-Score System:** Store the highest scores and display them on the game-over screen.
- **Sound Effects:** Add sounds for block movement, rotation, line clearing, and game over.

- **Better Animation:** Add a small animation when a complete line is removed.
- **Multiple Game Modes:** Add different difficulty levels and game modes.
- **Improved User Interface:** Add buttons and graphical menus instead of relying only on keyboard commands.
- **Background Music:** Add background music with an option to enable or disable it.
- **Modern Graphics:** Improve the block design using textures, lighting effects, and more advanced OpenGL techniques.
- **Save/Load Feature:** Allow the player to save the current game state and continue later.

### **Q5. Would you recommend this type of project in CG lab? Be honest with justifications.**

Yes, I would recommend this type of project for a Computer Graphics laboratory because it connects theoretical graphics concepts with an interactive application.

Instead of only implementing individual algorithms such as rotation or translation separately, the project combines several concepts into one working system. For example, block movement uses translation, block rotation uses rotational transformation, the board uses line and shape drawing, and collision detection uses coordinate and boundary calculations.

The project also makes it easier to understand why these concepts are important. When a block rotates incorrectly or moves outside the board, the graphical result immediately shows the problem. This makes debugging and learning more practical.

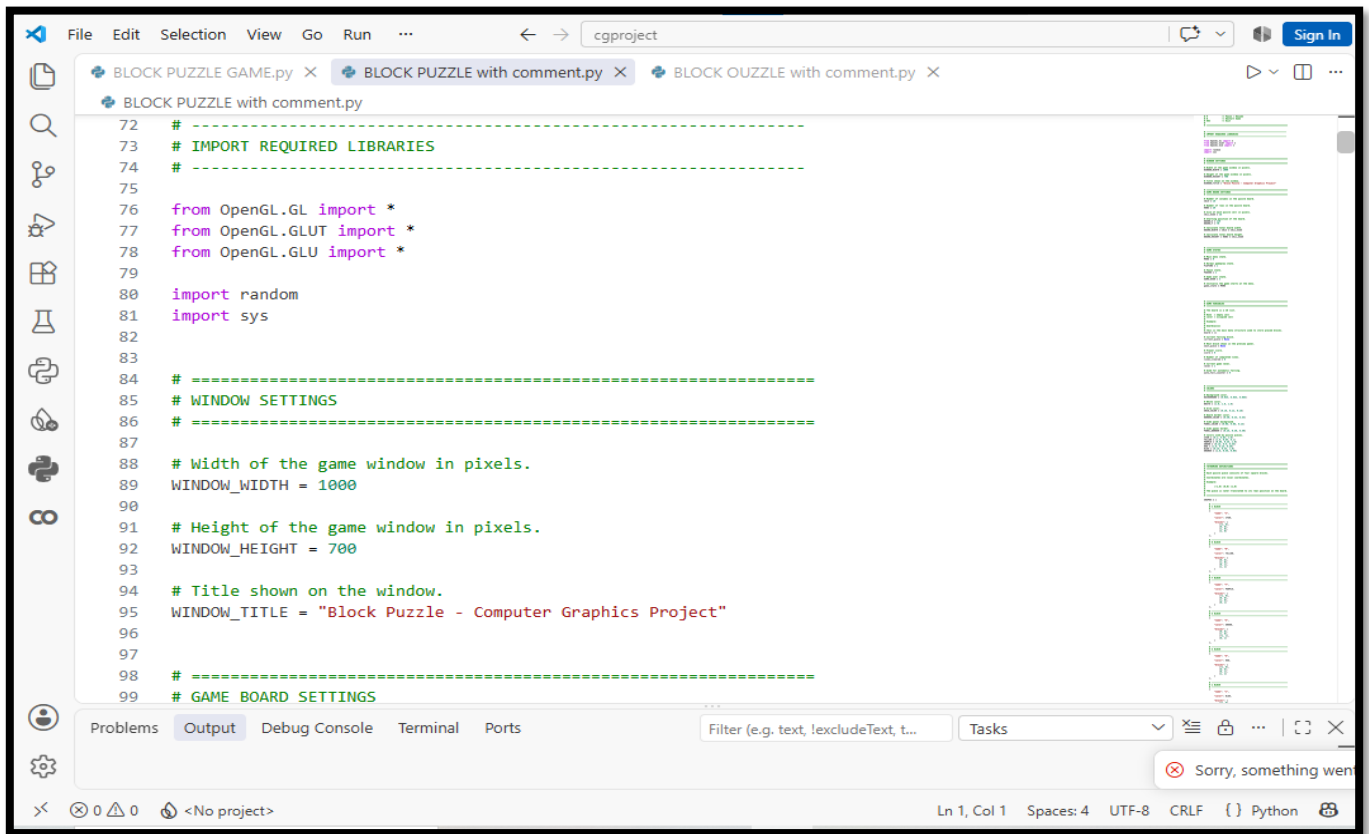
However, I have some honest suggestions. Setting up **PyOpenGL and GLUT on Windows** can take some time, especially for students who are using OpenGL for the first time. A short setup guide or laboratory session would make the process easier.

Another limitation is that the project uses traditional OpenGL/GLUT techniques. Modern graphics programming commonly uses shader-based OpenGL, Vulkan, DirectX, or game engines. Therefore, students should understand that the purpose of this project is mainly to learn the **fundamental Computer Graphics concepts**, rather than to represent a modern commercial game engine

### **Future Improvements**

- Start menu and instructions screen.
- Sound effects and background music.
- Improved block animation and visual effects.
- High-score saving.
- Multiple difficulty levels.
- Improved rotation near board boundaries.
- Ghost/landing-position preview.
- Touch or game-controller support.
- More advanced themes and visual effects.
- Standalone executable packaging.

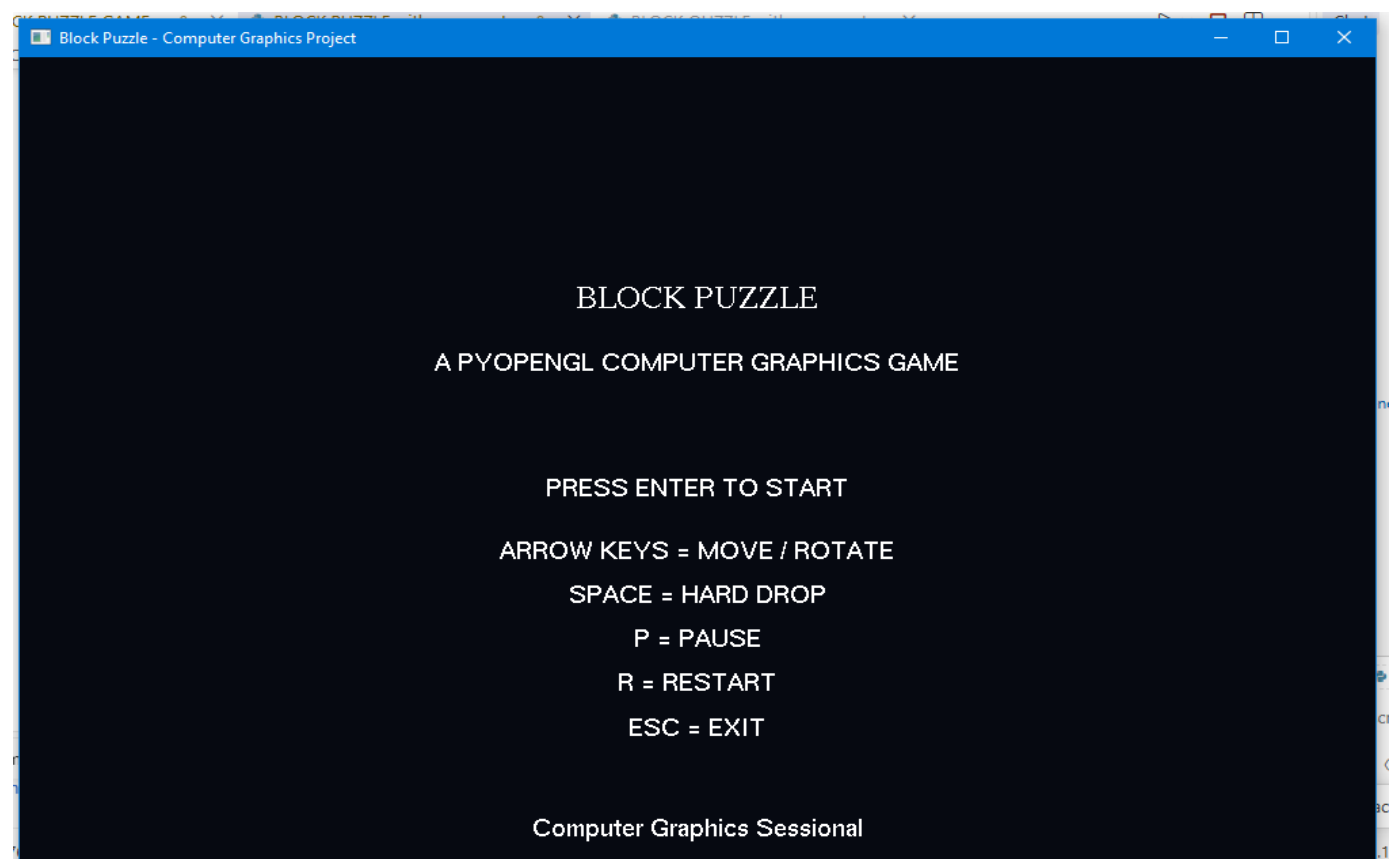
## Screenshots

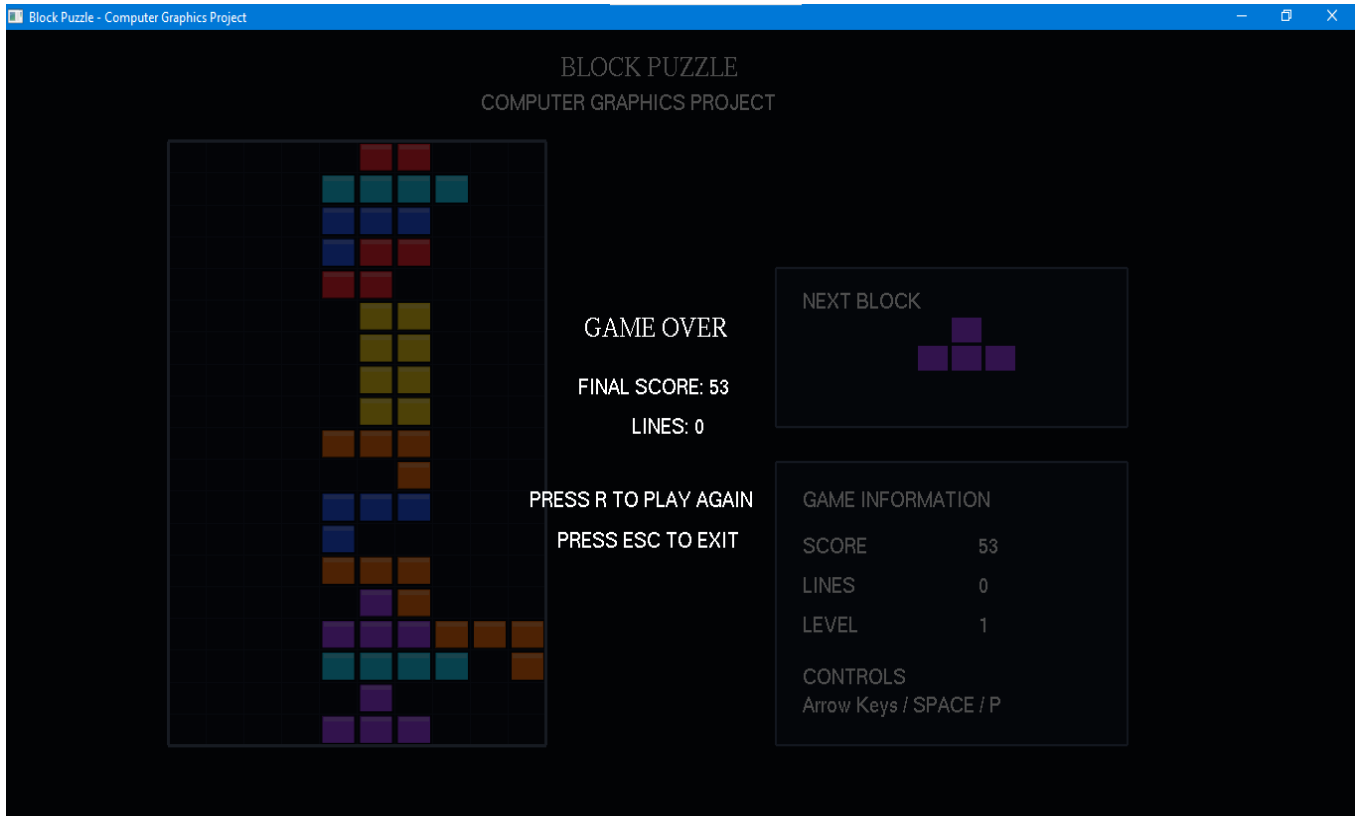
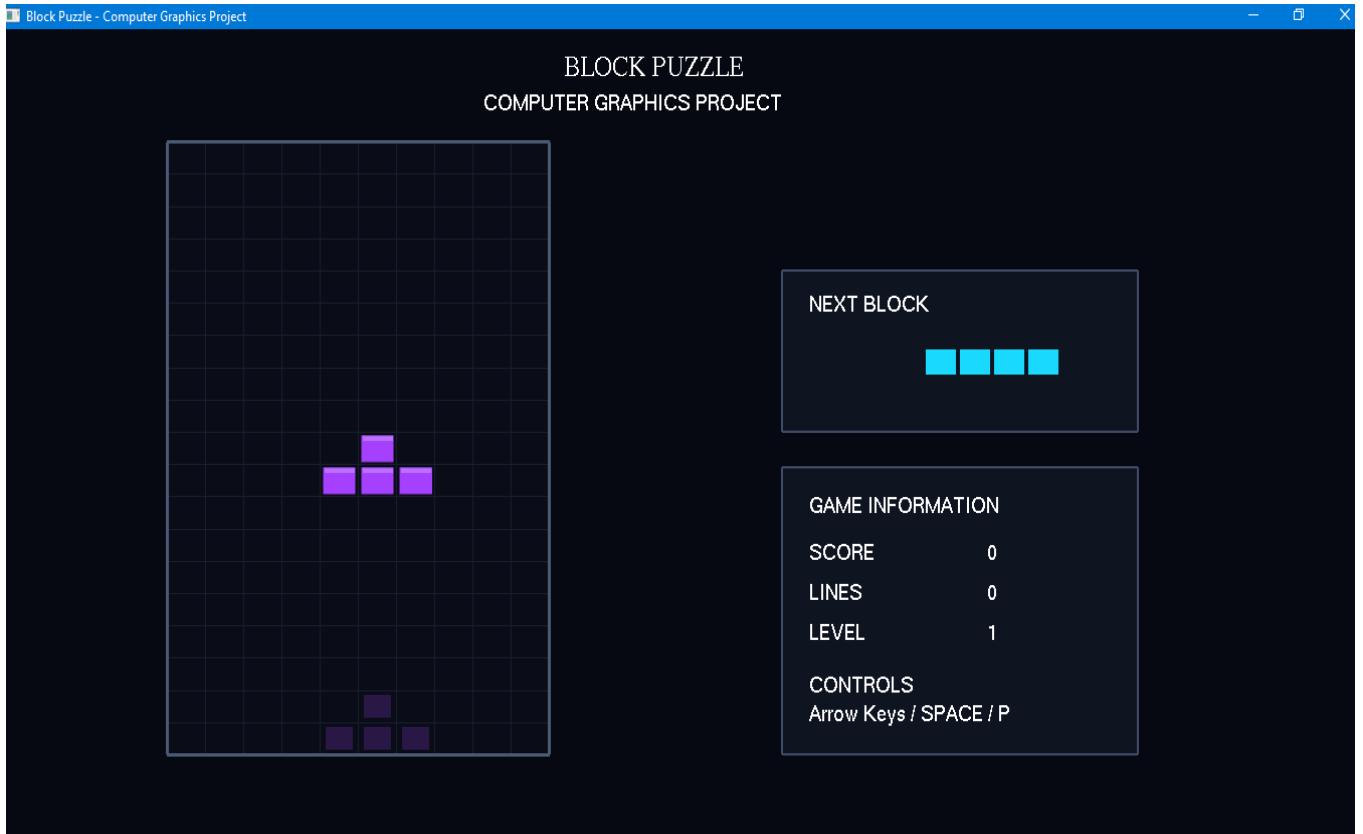


The screenshot shows a code editor with three tabs: "BLOCK PUZZLE GAME.py", "BLOCK PUZZLE with comment.py", and "BLOCK OUZZLE with comment.py". The active tab is "BLOCK PUZZLE with comment.py", which displays the following Python code:

```
72 # -----  
73 # IMPORT REQUIRED LIBRARIES  
74 # -----  
75  
76 from OpenGL.GL import *  
77 from OpenGL.GLUT import *  
78 from OpenGL.GLU import *  
79  
80 import random  
81 import sys  
82  
83  
84 # =====  
85 # WINDOW SETTINGS  
86 # =====  
87  
88 # Width of the game window in pixels.  
89 WINDOW_WIDTH = 1000  
90  
91 # Height of the game window in pixels.  
92 WINDOW_HEIGHT = 700  
93  
94 # Title shown on the window.  
95 WINDOW_TITLE = "Block Puzzle - Computer Graphics Project"  
96  
97  
98 # =====  
99 # GAME BOARD SETTINGS
```

The editor's interface includes a menu bar (File, Edit, Selection, View, Go, Run, ...), a toolbar with icons for file operations, a sidebar with icons for project management, and a bottom panel with tabs for Problems, Output, Debug Console, Terminal, and Ports. A status bar at the bottom shows "Ln 1, Col 1", "Spaces: 4", "UTF-8", "CRLF", and "Python".





## Conclusion

The Block Puzzle Game demonstrates how Computer Graphics concepts can be applied to a real interactive application. The project uses line and shape drawing, 2D transformations, color filling, Cohen-Sutherland clipping, and Bezier curves. It also combines graphics with collision detection, keyboard interaction, animation timing, scoring, level progression, and game-state management.

Through this project, I gained practical experience in converting Computer Graphics theory into a working game prototype. The project also connects directly with my selected career track in Game / Graphics Industry or Freelancing. It provides a foundation for future work involving graphics programming, animation, game engines, and interactive applications.

## GitHub Repository Link:

[https://github.com/mohsinshah2060/cg\\_project-1.git](https://github.com/mohsinshah2060/cg_project-1.git)

## References

- OpenGL documentation
- PyOpenGL documentation
- Python documentation
- GLUT documentation
- Any tutorial/**documentation you actually us**